Failure (a crashing tool, and a bad description) — Hints

> Back to the exercise · Hint 1 · Hint 2 · Solution

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper LangChain/Python). Read Basic first even if you already know this — it's the fastest way to spot exactly what Intermediate adds.

- Hint 1 — The idea, and the exact pieces
- Hint 2 — The plan, and almost the whole thing

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 1 — The idea, and the exact pieces

Basic Version

This exercise has two separate halves — do them one at a time, don't mix them together.

Half 1 — the crashing tool: write a tool that raises an exception on purpose (like dividing by zero, or raising `ValueError` directly). First, call it and let it crash your whole program — see that happen once. Then wrap the body in `try/except` and return a plain error string instead of letting it raise, and see the model actually receive that string as the tool's result.

Half 2 — the bad description: write a tool with a genuinely vague, unhelpful description (just "gets data" or similar). Run an ambiguous prompt against it 5 times alongside a second tool, note the split, then rewrite the description to be precise, and run the same prompt 5 more times. Compare the two splits.

Things to use:

- A tool whose body can raise (`1 / 0`, or a manual `raise ValueError("boom")`).
- `try/except Exception as e: return f"Error: {e}"` inside the tool.
- Two versions of the same tool's docstring: a vague one, then a precise one.

Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Half 1: first, watch it actually crash — call your failing tool function directly, unwrapped, and let the exception happen. You want to have actually seen the raw traceback once before you hide it, so "catching the error" doesn't feel like skipping a step. Then the key line is `return f"Error: {e}"` inside an `except` block, instead of letting the exception propagate. This is a deliberate design choice — the tool's "successful" return type and its "error" return type should look the same to the code path that reads the result (both are just strings the model can react to), because the model has no other way to see what happened. After catching the error and returning a string, pass that string back into the conversation as the tool's result, and ask the model to respond — check that it reacts sensibly to the error text instead of treating it as a real answer.

Half 2: reuse the two ambiguous tools from the Intermediate ambiguity exercise (or write two new ones) — give one of them a genuinely bad description, run your existing 5-run trial function against it, then edit only the description (not the function body), and run the exact same trial again. Print each run's pick both before and after, and count the split as a ratio (like "4/5 picked tool A") rather than eyeballing it. Write out, for Half 2, exactly what you're going to hold constant between the "before" and "after" runs — the comparison only means something if everything else stays fixed except the description text.

Difference between Basic and Intermediate: same two halves (catch instead of crash, measure a description fix) at 2 completeness levels — Basic gets both working once, Intermediate sends the error back to the model as a tool message and prints its real reaction, and gives both description versions the same tool name with `@tool("get_weather")`, so the comparison only changes the docstring. Worth knowing this version treats every exception the same way, and trusts a single 5-run comparison — a real, kept tool would want to separate expected failures from genuine bugs, and a bigger sample before acting on the result.

Back to the exercise · Hint 1 · Hint 2 · Solution

Hint 2 — The plan, and almost the whole thing

Basic Version

```
Half 1:
    define a tool that raises an error when called
    call it directly, unwrapped -> watch it crash

    wrap the body in try/except
    on error, return "Error: " + the error message

    register it on a model, ask something that triggers it
    feed the tool's error string back to the model
    print how the model responds to the error

Half 2:
    write a tool with a vague description
    run a 5-run selection trial -> record the split

    change only the description to be precise
    run the same 5-run trial again -> record the new split

    print both splits side by side
```

Here's the tricky part for Half 1 — turning a crash into a clean returned string:

```
# tool_selection_practice.py — Failure section
from langchain_core.tools import tool

@tool
def flaky_divide(a: int, b: int) -> str:
    """Divide a by b."""
    try:
        result = a / b
        return str(result)
    except Exception as e:
        return f"Error: {e}"
```

For Half 2, the trickiest part is making the "before" and "after" comparison fair: use the exact same trial function, the exact same prompt, and change nothing but the docstring. Finish both halves yourself, then check the Solution.

> Back to the exercise · Hint 1 · Hint 2 · Solution

Intermediate Version

Two small pieces to get you unstuck.

Half 1 — send the error back, the same growing-list idea as Doc04's memory exercise:

```
# tool_selection_practice.py — Failure section
messages = [{"role": "user", "content": "Use the tool to divide 10 by 0"}]
response = model_with_tools.invoke(messages)
messages.append(response)
for call in response.tool_calls:
    result = flaky_divide.invoke(call["args"])
    messages.append(
        {"role": "tool", "content": result, "tool_call_id": call["id"]}
    )
final = model_with_tools.invoke(messages)
```

Half 2 — the same tool name for both versions, so only the description differs:

```
@tool("get_weather")
def vague_weather(city: str) -> str:
    """gets data"""
    return f"Sunny in {city}"
```

Write `precise_weather` the same way with a precise docstring. Another way to keep it fair: a factory `make_weather_tool(description)` that defines one `get_weather`, sets `get_weather.__doc__ = description`, and returns `tool(get_weather)`. Try finishing both halves yourself before looking at the full Solution.

Back to the exercise · Hint 1 · Hint 2 · Solution

Full solution: Show me the solution